出版者的话

文艺复兴以降，源远流长的科学精神和逐步形成的学术规范，使西方国家在自然科学的各个领域取得了垄断性的优势；也正是这样的传统，使美国在信息技术发展的六十多年间名家辈出、独领风骚。在商业化的进程中，美国的产业界与教育界越来越紧密地结合，计算机学科中的许多泰山北斗同时身处科研和教学的最前线，由此而产生的经典科学著作，不仅擘划了研究的范畴，还揭橥了学术的源变，既遵循学术规范，又自有学者个性，其价值并不会因年月的流逝而减退。

近年，在全球信息化大潮的推动下，我国的计算机产业发展迅猛，对专业人才的需求日益迫切。这对计算机教育界和出版界都既是机遇，也是挑战；而专业教材的建设在教育战略上显得举足轻重。在我国信息技术发展时间较短、从业人员较少的现状下，美国等发达国家在其计算机科学发展的几十年间积淀的经典教材仍有许多值得借鉴之处。因此，引进一批国外优秀计算机教材将对我国计算机教育事业的发展起积极的推动作用，也是与世界接轨、建设真正的世界一流大学的必由之路。

机械工业出版社华章图文信息有限公司较早意识到“出版要为教育服务”。自1998年开始，华章公司就将工作重点放在了遴选、移译国外优秀教材上。经过几年的不懈努力，我们与Prentice Hall，Addison-Wesley，McGraw-Hill，Morgan Kaufmann等世界著名出版公司建立了良好的合作关系，从它们现有的数百种教材中甄选出Tanenbaum，Stroustrup，Kernighan，Jim Gray等大师名家的一批经典作品，以“计算机科学丛书”为总称出版，供读者学习、研究及庋藏。大理石纹理的封面，也正体现了这套丛书的品位和格调。

“计算机科学丛书”的出版工作得到了国内外学者的鼎力襄助，国内的专家不仅提供了中肯的选题指导，还不辞劳苦地担任了翻译和审校的工作；而原书的作者也相当关注其作品在中国的传播，有的还专诚为其书的中译本作序。迄今，“计算机科学丛书”已经出版了近百个品种，这些书籍在读者中树立了良好的口碑，并被许多高校采用为正式教材和参考书籍，为进一步推广与发展打下了坚实的基础。

随着学科建设的初步完善和教材改革的逐渐深化，教育界对国外计算机教材的需求和应用都步入一个新的阶段。为此，华章公司将加大引进教材的力度，在“华章教育”的总规划之下出版三个系列的计算机教材：除“计算机科学丛书”之外，对影印版的教材，则单独开辟出“经典原版书库”；同时，引进全美通行的教学辅导书“Schaum's Outlines”系列组成“全美经典学习指导系列”。为了保证这三套丛书的权威性，同时也为了更好地为学校和老师们服务，华章公司聘请了中国科学院、北京大学、清华大学、国防科技大学、复旦大学、上海交通大学、南京大学、浙江大学、中国科技大学、哈尔滨工业大学、西安交通大学、中国人民大学、北京航空航天大学、北京邮电大学、中山大学、解放军理工大学、郑州大学、湖北工学院、中国国家信息安全测评认证中心等国内重点大学和科研机构在计算机的各个领域的著名学者组成“专家指导委员会”，为我们提供选题意见和出版监督。

这三套丛书是响应教育部提出的使用外版教材的号召，为国内高校的计算机及相关专业

的教学度身订造的。其中许多教材均已为M. I. T.，Stanford，U.C. Berkeley，C. M. U. 等世界名牌大学所采用。不仅涵盖了程序设计、数据结构、操作系统、计算机体系结构、数据库、编译原理、软件工程、图形学、通信与网络、离散数学等国内大学计算机专业普遍开设的核心课程，而且各具特色——有的出自语言设计者之手、有的历经三十年而不衰、有的已被全世界的几百所高校采用。在这些圆熟通博的名师大作的指引之下，读者必将在计算机科学的宫殿中由登堂而入室。

权威的作者、经典的教材、一流的译者、严格的审校、精细的编辑，这些因素使我们的图书有了质量的保证，但我们的目标是尽善尽美，而反馈的意见正是我们达到这一终极目标的重要帮助。教材的出版只是我们的后续服务的起点。华章公司欢迎老师和读者对我们的工作提出建议或给予指正，我们的联系方法如下：

电子邮件：hzedu@hzbook.com
联系电话：（010）68995264
联系地址：北京市西城区百万庄南街1号
邮政编码：100037

专家指导委员会

（按姓氏笔画顺序）

尤晋元	王　珊	冯博琴	史忠植	史美林
石教英	吕　建	孙玉芳	吴世忠	吴时霖
张立昂	李伟琴	李师贤	李建中	杨冬青
邵维忠	陆丽娜	陆鑫达	陈向群	周伯生
周立柱	周克定	周傲英	孟小峰	岳丽华
范　明	郑国梁	施伯乐	钟玉琢	唐世渭
袁崇义	高传善	梅　宏	程　旭	程时端
谢希仁	裘宗燕	戴　葵		

读者评论

每个Java程序员都应该反复研读《Think in Java》，并且随身携带以便随时参考。书中的练习颇具挑战性，而有关集合的章节已臻化境！本书不仅帮助我通过了Sun Certified Java Programmer考试，而且它还是我遇到Java问题时，求助的首选书籍。

Jim Pleger, Loudoun郡（弗吉尼亚）政府

这本书比我见过的所有Java书都要好得多。循序渐进……非常完整，并搭配恰到好处的范例，睿智而不呆板的解说……这使本书的品质比别的书“超出了一个数量级”。与其他Java书相比，我发现本书考虑非常周全、前后一致、理性坦诚、文笔流畅、用词准确。恕我直言，这是一本学习Java的理想书籍。

Anatoly Vorobey, 以色列海法Technion大学

在我所见过的程序设计指南中（无论何种语言），这绝对是最好的一本。

Joakim Ziegler, FIX系统管理员

感谢您这本精彩的、令人愉快的Java书。

Dr. Gavin Pillay, 登记员，南非爱德华八世医院

再次感谢您这本杰出的书。作为一名不用C语言的程序员，我曾经感到（学习Java）步履维艰，但是您的书让我一目了然。能够一开始就理解底层的概念和原理，而不是通过反复试验来自己建立概念模型，真是太棒了。我希望能在不久的将来参加您的讨论课。

Randall R. Hawley, 自动化工程师, Eli Lilly公司

我见过的计算机著作中，这是最好的一本。

Tom Holland

这是我读过的编程语言书中最棒的一本……有关Java的书中最棒的一本。

Ravindra Pai, Oracle 公司, SUNOS 产品线部门

我见过的最好的Java书！您做了一项了不起的工作。您的深度令人赞叹，出版的时候，我一定会购买一本。我从1996年10月就开始学习Java，其间也读过好几本这方面的书，但我觉得您这本才是“必读书”。最近几个月，我一直集中精力于一个完全用Java开发的产品。您的书帮我夯实了某些不牢固的知识点，并拓展了我的知识面。我甚至在面试签约者时引用书中的内容，作为参考的依据。通过问一些我从书中学到的知识，来判断他们对Java的理解程度（例如，数组与Vector的区别）。您的书真是伟大！

Steve Wilkinson, 资深专家, MCI 电信公司

伟大的书。迄今为止我见过的最佳Java书籍。

Jeff Sinclair, 软件工程师, Kestral 计算技术公司

感谢您的《Thinking in Java》。早就应该有人把仅仅介绍语言的教程编写成富有思想、分析

透彻的入门指南，而不是局限于“某个公司”的语言。我阅读过许多这方面的书，但只有您和Patrick Winston的作品给我印象深刻。我已经向客户推荐这本书。再次谢谢您。

Richard Brooks, Java 咨询顾问，达拉斯Sun专业服务部门

Bruce，您的书真是太棒了！您的讲解清晰明确。通过这本迷人的书，我获得了大量Java知识。练习题也同样令人着迷，它们对巩固各章阐述的知识起到了很好的效果。我期待您的更多作品。对您的这本著作致以谢意。阅读了《Thinking in Java》之后，我的代码质量大有改善。为此我要感激您，我相信，维护我的代码的程序员同样也会感激您。

Yvonne Watkins, Discover 技术公司

其他书籍只涵盖Java 的WHAT（探讨语法和相关程序库），或者只包含Java的HOW（实际的程序范例）。《Thinking in Java》则是我知道的书籍中唯一对Java的WHY做出讲解的一本。为什么要这样设计，为什么它会那样运作，为什么有时候会发生问题，为什么它在某些方面比C++好而某些方面不会。虽然它在教授程序语言的WHAT和HOW方面也很成功，但《Thinking in Java》更是爱钻研者的首选Java书籍。

Robert S. Stephenson

感谢您写了一本伟大的书。我越看越喜欢。我的学生也很喜欢。

Chuck Iverson

我要赞美您在《Thinking in Java》一书上的表现。正是有了您这样的人，才使得因特网充满前景，而我想感谢您的付出与努力。真是感激不尽。

Patrick Barrell, Network Officer Mamco, QAF Mfg. Inc.

我真的非常感激您的热情与您的作品。我下载了你的在线书籍的每一个修订版本，我正在深入钻研语言，并探索那些以前从来不敢碰的内容（对于C#、C++、Python和Ruby也有作用）。我至少还有15本Java书（为了工作，我还要掌握JavaScript和PHP语言），并且订阅了《Dr. Dobbs》、《JavaPro》、《JDJ》、《JavaWorld》等杂志。在深入钻研Java（包括Java企业版）之后，我对您的书更加尊敬。它的确是一本有思想的书。我订阅了您的邮件列表，并希望有一天，我所探讨和解决的问题能被您扩展到解题指导中（我将购买解题指导！）。同时，非常感激。

Joshua Long, www.starbuxman.com

市面上的Java书籍，大多比较适合初学者。它们大多数也就只具备基础内容，范例也大同小异。在我见过的富有思想性并讲解高级主题的书籍中，您的是最好的。快点出版吧！……鉴于《Thinking in Java》带给我的深刻印象，我也购买了《Thinking in C++》。

George Laframboise, LightWorx 技术咨询公司

关于您的《Thinking in C++》（我工作的时候，它总在书架上占据最显眼的位置），我曾经写信告诉过您我对它的喜爱。现在，我通过您的电子书仔细钻研Java，我还得说“我喜欢！”本书内容广博，讲解详细，阅读起来不像是无味的教科书。您的书中涵盖了Java 开发工作中最重要、却很少被提及的概念——“原理”。

Sean Brady

我同时用Java和C++进行开发，您的这两本书是我的救星。如果我被某个问题难住了，我知道可以靠您的书来：a) 清楚地解释原因；b) 找到符合我所遇问题的具体例子。我还没找到另一

位能令我如此反复热情推荐的作者（如果有人愿意听我推荐的话）。

Josh Asbury, A^3 软件咨询公司，辛辛那提，俄亥俄

您的例子不仅清楚，而且容易理解。Java中的许多重要细节您都考虑到了，这些内容在编排较差的Java文档中很难找到。您假设程序员已经具有了基本知识，这就节约了读者的时间。

Kai Engert, 德国Innovative 软件公司

我是《Thinking in C++》一书的忠实书迷，我已经将它推荐给了我的同事们。当我读完您的Java书籍电子版时，我觉得，您总能保持高水准的写作水平。感谢您！

Peter R. Neuwald

写得非常好的Java书……我认为您在此书上取得了非常出色的成就。作为芝加哥地区Java兴趣小组的领导人，我已经多次在我们最近的聚会中赞扬您的这本书和您的网站。我想将《Thinking in Java》作为我们每月聚会讨论的主要内容。这样我们可以在聚会中对书中的章节进行复习和讨论。

Mark Ertes

顺便提一下，《Thinking in Java 2nd Edition》俄语版依旧畅销。阅读此书已经与学习Java成为同义词，真是太好了。

Ivan Porty（《Thinking In Java 2nd Edition》俄语版的译者及出版商）

对于您的辛勤工作，我由衷感激。您的书是佳作，我将这本书推荐给我们这儿的使用者和博士班学生。

Hugues Leroy // Irisa-Inria Rennes France,
Head of Scientific Computing and Industrial Tranfert

虽然我只读了约40 页的《Thinking in Java》，却已经发现本书是我所见过的讲述最为清晰、编排最为合理的程序设计书籍……作为一名作者，我可能会有些挑剔。我已经订购了《Thinking in C++》，迫不及待地想钻研一番。对于程序设计，我还算是新手，因此事事都得学习。这不过是一篇向您的绝佳作品致谢的简短书信。在痛苦地遍览大多数语言艰涩、内容散乱的计算机书籍（包括那些有着极佳口碑的书籍）后，我对计算机书籍的阅读热情一度消退。不过，现在我又重拾信心。

Glenn Becker, Educational Theatre Association

感谢您提供了这么一本精彩的书。当我遇到那些令人困惑的Java 和C++问题时，这本书对我最终理解问题提供了极大帮助。阅读您的书令人如沐春风。

Felix Bizaoui, Twin Oaks Industries, Louisa, Va.

对于这部优秀的作品，我必须向您道贺。鉴于阅读《Thinking in C++》的经验，我决定读一读《Thinking in Java》，而事实证明它的确未让人失望。

Jaco van der Merwe, 南非DataFusion系统公司软件专家

本书无疑是我所见过的最佳的Java书籍之一。

E.F. Pritchard, 英国剑桥动画系统公司高级软件工程师

、 您的书使那些我曾经读过或草草翻过的Java 书显得更加无用、该骂。

Brett g Porter, Art & Logic公司高级程序员

我阅读您这本书已经一两个星期了。与以前我曾读过的Java 书籍比较，您的书似乎更能给我一个绝佳的开始。我已经把此书推荐给我的朋友们，他们对此书也评价甚高。对于您写出的这本著作，请接受我的恭喜。

Rama Krishna Bhupathi, 加州圣何塞TCSI 公司软件工程师

只是很想告诉您，您这本书是多么杰出的作品。我已将它作为公司内部Java工作的主要参考书。我发现目录的安排恰如其分，可以很快找到需要的章节。能够看到这么一本既不拿API炒冷饭，也不把程序员当傻瓜的书，真是太棒了。

Grant Sayer, 澳大利亚Ceedata 系统私人有限公司，Java组件组长

哇！这是一本可读性强、极富深度的Java书籍。市面上已经有太多质量低劣的Java 书籍。其中虽然也有少数不错的，但在看过您的大作之后，我认为它当然是最好的。

John Root, 伦敦社会安全局Web开发人员

我才刚开始阅读《Thinking in Java》。我想它一定相当不错，因为我很喜欢《Thinking in C++》（我以一名熟悉C++、同时希望提升自身能力的程序员身份来阅读这本书）。尽管我不太熟悉Java，但本书想必能令我满意。您是一位伟大的作家。

Kevin K. Lewis, ObjectSpace公司技术专家

我想这是一本了不起的书。我所有的Java知识都学自这本书。感谢您让大家可以从Internet上免费取得这本书。如果没有您的付出，我至今恐怕仍然对Java一无所知。本书最棒的一点，莫过于它同时也说明了Java 不好的一面，而不像那些商业宣传资料。您的表现真是优秀。

Frederik Fix, 比利时

我始终热中读您的著作。几年以前，当我开始学习C++时，是《C++ Inside & Out》带领我进入C++的迷人世界。那本书帮助我得到了更好的机会。现在，为了更进一步钻研知识，我兴起了学习Java 的念头，我无意中又碰见了《Thinking in Java》。毫无疑问，我认为自己不再需要其他书籍。它是那么的令人难以置信。阅读此书的过程，就像重新发掘自我一样。我学习Java至今只有一个月，现在对Java的体会日益加深，这一切都不得不由衷感谢您。

Anand Kumar S., 印度Computervision公司软件工程师

您的书作为综合性的导论，是如此出色。

Peter Robinson, 剑桥大学计算机实验室

在帮助我学习Java的书籍中，这一本显然是最好的。我只是想让您知道，我觉得自己能够读到这本书是多么幸运。谢谢！

Chuck Peterson, IVIS 国际公司Internet产品线产品组长

了不起的一本书。自从我开始学习Java，这已经是第三本了。目前我大概阅读了三分之二，并打算把它读完。我能够找到这本书，是因为这本书被用于Lucent 技术公司的某些内部课程，而且有个朋友告诉我这本书可以在网络上找到。很棒的作品。

Jerry Nowlin, Lucent 技术公司MTS部门

在我所读过的六本Java书籍中，您的《Thinking in Java》显然最好，也最清晰易懂。

Michael Van Waas博士, TMR Associates公司总裁

感谢您的《Thinking in Java》。您的作品真是精彩！更不必说它可以免费从网络下载了！作为一名学生，我觉得您的书籍是无价之宝（我也有一本《C++ Inside & Out》，它同样是一本伟大的C++书籍），因为您的书不仅教导我应该怎么做，也教导我这么做的原因所在，这一点对C++或Java学习者建立起坚固基础非常重要。我有许多和我一样喜爱程序设计的朋友，我也对他们提起您的书。他们觉得真是太棒了！再次谢谢您！顺道一提，我是印度尼西亚人，就住在“爪哇”（Java）。

Ray Frederick Djajadinata, 印度尼西亚雅加达Trisakti 大学学生

单是将作品免费放在网络上这种气度，就令我震惊不已。我想，我应该让您知道，对于您的工作，我是多么感激与尊敬。

Shane LeBouthillier, 加拿大 Alberta大学计算机工程系学生

我得告诉您，每个月我都在期待您的专栏。作为面向对象程序设计领域的新手，我要感谢您花在那些基础主题上的时间和思考。我已经下载了您的这本书，而且我一定会在本书出版的时候购买一本。感谢您对我的帮助。

Dan Cashmer, B. C. Ziegler公司

能够完成这么了不起的作品，恭喜您。开始，我偶然发现了《Thinking in Java》的PDF版本。甚至在我读完之前，我又跑到书店找到了《Thinking in C++》。我已经在计算机领域工作了八年多，做过顾问、软件工程师、教师/教练，最近则从事自由职业。所以我觉得自己也算是见多识广了（注意，不是“无所不知”，而只是“见多识广”）。不过，这些书使得我的女朋友称我为“呆子”。我并不反对，只不过我发现自己已经远远超过这个阶段。我发现自己如此喜爱这两本书，我以前接触过或购买过的其他计算机书籍，都无法与之相比。这两本书都有极佳的写作风格，对于每个新主题都有很好的介绍，书中充满了睿智的见解。干得好。

Simon Goland, simonsez@smartt.com, Simon Says 咨询公司

我得说，您的《Thinking in Java》真是了不起。它正是我要找的那种书。尤其那些讨论优秀与拙劣的Java 软件设计的章节，完全就是我想要的。

Dirk Duehr, 德国贝塔斯曼集团 Lexikon 公司

感谢您写了两本著作：《Thinking in C++》和《Thinking in Java》。在面向对象程序设计的学习过程中，您带给我巨大帮助。

Donald Lawson, DCL Enterprises公司

感谢您花时间来撰写这么一本很有用的Java书籍。如果是教学让您明白了某些事情的话，到如今您一定极为满意自己的成就。

Dominic Turner, GEAC Support

我曾读过的最棒的Java 书籍——我真的读过不少。

Jean-Yves MENGANT, 法国巴黎 NAT-SYSTEM公司首席系统架构师

《Thinking in Java》涵盖全面，讲解清晰。本书极易阅读，而且程序代码也是如此。

Ron Chan博士，匹兹堡 Expert Choice公司

您的书真好。我读过许多程序设计书籍，但是这本书中您对程序设计的深刻见解依然深深触动了我。

Ningjian Wang, Vanguard 集团信息系统工程师

《Thinking in Java》是一本既优秀，又容易阅读的书籍。我向所有的学生推荐它。

Dr. Paul Gorman, 新西兰 Otago大学计算机科学系

依靠您的书，我现在已经理解了面向对象程序设计的含义……我相信，Java比Perl更直接，甚至更容易。

Torsten Römer, Orange 丹麦公司

您打破了“天下没有白吃的午餐”这句谚语。不是那种施舍性质的午餐，而是连美食家都觉得美味的午餐。他们都会为此感激您。

Jose Suriol, Scylax 公司

感谢有机会看到这本书成为一部杰作！在这个主题上，本书绝对是我所读过的最佳书籍。

Jeff Lapchinsky, Net Results 技术公司程序员

您的书简明扼要，容易理解，而且读起来充满乐趣。

Keith Ritchie, KL 集团公司Java研发组

确实是我所读过的最好的Java 书籍！

Daniel Eng

生平所见最好的Java 书籍！

Rich Hoffarth, West 集团高级架构师

感谢您带来了如此精彩的一本好书。通读各个章节带给我极大的乐趣。

Fred Trimble, Actium 公司

您一定掌握了艺术的精髓，使我们得以循序渐进地成功掌握细节知识。您也让学习过程变得非常简单，同时令人愉快。感谢您这本真正精彩的指南。

Rajesh Rau, 软件顾问

《Thinking in Java》撼动了整个自由世界！

Miko O'Sullivan, Idocs 公司总裁

关于《Thinking in C++》[⊖]

最好的书！1995年《Software Development》杂志Jolt大奖得主！

“本书成就非凡。您应该在书架上也摆一本。其中讨论输入、输出流的章节，在我所见过的有关此主题的论著中，它是表述最全面、也最容易理解的。”

Al Stevens，《Dr. Dobbs Journal》的特约编辑

“对于如何重新认识面向对象程序的构造，Eckel的这本书是唯一能做出如此清晰解释的书籍。同时，它也是透彻讲解C++的优秀教程。”

Andrew Binstock，《Unix Review》的编辑

“Bruce对C++的洞察力，不断令我感到惊讶。《Thinking in C++》则是他迄今为止所有绝妙想法的最佳合集。有关C++的种种难题，如果您需要清楚的解答，请买下这本杰作。”

Gary Entsminger，《The Tao of Objects》的作者

《Thinking in C++》耐心而系统地对C++种种特性的使用时机与方式进行了探讨。包括：内联函数、引用、操作符重载、继承、动态对象。也包括了许多高级主题，比如模板、异常、多重继承的恰当用法。对这些交织在一起，最后形成了Eckel对对象和程序设计的独特看法。它是每个C++开发者书架上的必备好书。如果您正以C++从事严肃的开发工作，那么《Thinking in C++》是您的必备书籍之一。

Richard Hale Shaw，《PC Magazine》的特约编辑

⊖ 本书第1卷与第2卷的中文版与英文版均已由机械工业出版社出版。——编辑注

译 者 序

时隔两年多，《Java编程思想（第4版）》的中文版又要和广大Java程序员和爱好者们见面了。这是Java语言本身不断发展和完善的必然要求，也是本书作者Bruce Eckel孜孜不倦的创作激情和灵感所结出的硕果。

《Java编程思想（第4版）》以Java最新的版本JDK5.0为基础，在第3版的基础上，添加了最新的语言特性，并且对第3版的结构进行了调整，使得所有章节的安排更加遵照循序渐进的特点，同时每一章的内容在分量上也都更加均衡，这使读者能够更加容易地阅读本书并充分了解每章所讲述的内容。在这里我们再次向Bruce Eckel致敬，他不但向我们展示了什么样的书籍才是经典书籍，而且还展示了经典书籍怎样才能精益求精，长盛不衰。

Java已经成为了编程语言的骄子。我们可以看到，越来越多的大学在教授数据结构、程序设计和算法分析等课程时，选择以Java语言为载体。这说明Java语言已经是人们构建软件系统时主要使用的一种编程语言。但是，掌握好Java语言并不是一件可以轻松完成的任务，如何真正掌握Java语言，从而编写出健壮的、高效的以及灵活的程序是Java程序员们面临的重大挑战。

《Java编程思想（第4版）》就是一本能够让Java程序员轻松面对这一挑战，并最终取得胜利的经典书籍。本书深入浅出、循序渐进地把我们领入Java的世界，让我们在不知不觉中就学会了用Java的思想去考虑问题、解决问题。本书不仅适合Java的初学者，更适合于有经验的Java程序员，这正是本书的魅力所在。但是，书中并没有涵盖Java所有的类、接口和方法，因此，如果你希望将它当作Java的字典来使用，那么显然就要失望了。

我们在翻译本书的过程中力求忠于原著，为了保持连贯性，对原书第3版中仍然保持不变的部分，我们对译文除了个别地方之外，也没做修改。对于本书中出现的大量的专业术语尽量遵循标准的译法，并在有可能引起歧义之处注有英文原文，以方便读者对照与理解。

全书由陈昊鹏翻译，郭嘉也参与了部分翻译工作。由于水平有限，书中出现错误与不妥之处在所难免，恳请读者批评指正。

译 者

2007年5月

译者简介

陈昊鹏，1975年生，2001年毕业于西北工业大学计算机科学与工程系，获工学博士学位，2004年从上海交通大学计算机科学与技术博士后流动站出站后，在上海交通大学软件学院软件工程中心任教至今。主要从事软件工程和分布式计算方面的研究。

主要翻译的译著有：

[1] Bruce Eckel. Java编程思想（第3版）[M]. 3版. 陈昊鹏，饶若楠，译. 北京：机械工业出版社，2005.

[2] Cay S. Horstmann，Gary Cornell. Java 2核心技术，卷II：高级特性（原书第7版）[M]. 陈昊鹏，王浩，姚建平，等译. 北京：机械工业出版社，2006.

[3] Ted Neward. Effective Enterprise Java中文版[M]. 陈昊鹏，薛翔，郭嘉，等译. 北京：机械工业出版社，2005.

[4] Ken Arnold，James Gosling，David Holmes. Java程序设计语言（第4版）[M]. 4版. 陈昊鹏，章程，张思博，等译. 北京：人民邮电出版社，2006.

[5] Joshua Bloch, Neal Gafter. Java解惑 [M]. 陈昊鹏，译. 北京：人民邮电出版社，2006.

题　献

献给 Dawn

前　言

一开始，我只是将Java看作“又一种程序设计语言”。从许多方面看，它也的确如此。

但随着时间流逝，以及对Java的深入研究，我渐渐发现，与我所见过的其他编程语言相比，Java有着完全不同的核心目的。

程序设计其实是对复杂性的管理：待解决问题的复杂性，以及用来解决该问题的工具的复杂性。正是这种复杂性，导致多数程序设计项目失败。在我所知的所有程序设计语言中，几乎没有哪个将自己的设计目标专注于克服开发与维护程序的复杂性[⊖]。当然，有些编程语言在设计决策时也曾考虑到复杂性的问题，然而，总是会有其他议题被认为更有必要加入到该语言中。于是不可避免地，正是这些所谓更必要的议题导致程序员最终“头撞南墙”。例如，C++选择向后兼容C（以便更容易吸引C程序员），以及具备C一样的高效率。这两点都是非常有益的设计目标，也确实促成了C++的成功，然而它们却暴露出更多的复杂性问题，而这也使得很多项目不得善终（你自然可以责怪程序员或者项目管理，但是，如果一种语言能够帮助你解决错误，那何乐而不为呢？）。再看一个例子，Visual Basic（VB）选择与Basic绑在一起，而Basic并未被设计为具备可扩展性的程序设计语言，结果呢，建立在VB之上的所有扩展都导致了无法维护的语法。还有Perl，它向后兼容awk、sed、grep，以及所有它打算替代的Unix工具，结果呢，人们开始指责Perl程序成了“不可阅读（write-only）的代码”（即，只要稍过一会儿，你就读不懂刚完成的程序了）。从另一个角度看，在设计C++、VB、Perl以及Smalltalk之类的程序设计语言时，设计师也都为解决复杂性问题做了某种程度的工作。并且，正是解决某类特定问题的能力，成就了它们的成功。

随着对Java的了解越来越深，Sun对Java的设计目标给我留下了最深刻印象，那就是：为程序员减少复杂性。用他们的话说就是：“我们关心的是，减少开发健壮代码所需的时间以及困难。”在早期，这个目标使得代码的运行并不快（Java程序的运行效率已经改善了），但它确实显著地缩短了代码的开发时间。与用C++开发相同的程序相比，采用Java只需一半甚至更少的开发时间。仅此一项，就已经能节约无法估量的时间与金钱了。然而Java并未止步于此。它开始着手解决日渐变得重要的各种复杂任务，例如多线程与网络编程，并将其作为语言特性或以工具库的形式纳入Java，这使得开发此类应用变得倍加简单。最终，Java解决了一些相当大的复杂性问题：跨平台编程、动态代码修改，甚至是安全的议题。它让你在面对其中任何一个问题时，都能从“举步维艰”到“起立鼓掌”。抛去我们都能看到的性能问题，Java确实非常精彩地履行了它的诺言：极大地提升程序员的生产率。

同时，Java正从各个方面提升人们相互通讯的带宽。它使得一切都变得更容易：编写程序，团队合作，创建与用户交户的用户界面，在不同类型的机器上运行程序，以及编写通过因特网通信的程序。

我认为，通讯变革的成果并不见得就是传输巨量的比特。我们所看到的真正变革是人与人

⊖ 不过，我相信Python语言非常接近该目标了。参见www.python.org。

之间的通讯变得更容易了：无论是一对一的通信，还是群体与群体之间，甚至整个星球之间的通信。我曾听闻，在足够多的人之间的相互联系之上，下一次变革将是一种全球意识的形成。Java说不定就是促进该变革的工具，至少，它所具备的可能性使我觉得，教授这门语言是非常有意义的一件事情。

Java SE5与SE6

本书的第4版得益于Java语言的升级。Sun起初称其为JDK1.5，稍后改作JDK5或J2SE5，最终Sun弃用了过时的“2”，将其改为Java SE5。Java SE5的许多变化都是为了改善程序员的体验。你将会看到，Java语言的设计者们并未完全成功地完成该任务，不过，总的来说，他们已经向正确的方向迈出了一大步。 2

新版的一个重要目标就是完整地吸收Java SE5/6的改进，并通过本书介绍以及应用这些变化。这意味着本书基本可以称之为“只限Java SE5/6”。并且，书中的多数代码并没有经过老版本的Java编译测试，所以如果你使用的是老版本的Java，编译可能会报错并中止。不过，我觉得这样利大于弊。

如果你不得不采用老版本的Java，我仍然为你在www.MindView.net提供了本书早期版本的免费下载。基于某些原因，我决定不提供本书当前版本的免费电子版。

Java SE6

本书是一个非常耗时的，且具有里程碑意义的一个项目。就在本书出版之前，Java SE6（代号野马mustang）已经发布了beta版。虽然Java SE6中的一些小变化，对书中的代码示例有一点影响，但其主要的改进对本书的绝大部分内容并没有影响。因为Java SE6主要关注于提升速度，以及改进一些（不在本书讨论范围之内）类库的特性。

本书中代码全部用Java SE6的一个发布候选版（RC）进行过测试，因此我不认为Java SE6正式发布时会有什么变化能够影响本书的内容。如果到时真的有什么重要的改变，我将更新本书中的代码，你可以通过www.MindView.net下载。

本书的封面已经指出，本书面向“Java SE5/6”。也就是说本书的撰写“面向Java SE5及其为Java语言引入的重大变化，同时也适用于Java SE6”。

第4版

为一本书写作新版时，作者最满意的是：把事情做得“恰如其分”。这是我从本书上一个版本发布以来所学到的东西。通常而言，这种见识正如谚语所云，“学习就是从失败中汲取教训。”并且，我也借机进行了一些修订。与往常一样，一个新的版本必将带来引人入胜的新思想。此时，新发现带来的喜悦，采用比以往更好的形式表达思想的能力，已经远远超过了可能引入的小错误。 3

这也是对不断在我脑中盘旋低语着的一种挑战，那就是让持有本书老版本的读者也愿意购买新的版本。这些促使着我尽可能改进，重写，以及重新组织内容，为热忱的读者们献上一本全新的，值得拥有的书。

改变

此版本中将不再包含以往本书中所携带的CD光盘。该CD中的重要部分《Thinking in C》的多媒体教程（由Chuck Allison为MindView创建），现在提供了可下载的Flash版本。该教程是为不

熟悉C语法的读者所准备的。虽然，本书用了两章对语法做了较为完整的介绍，然而对于没有相应背景知识的读者而言，这也许仍然不够。而《Thinking in C》正是为了帮助这些读者提升到必要的程度。

完全重写了“并发”这一章（以前称为“多线程”），以符合Java SE5并发类库的重大改变。它将为读者了解并发的核心思想打下基础。如果没有这些核心的基础知识，读者很难理解关于线程的更复杂的议题。我花了很多个月撰写这一章，深陷“并发”的地狱之中，最终，这一章不仅涵盖了基础知识，而且大胆地引入了一些高级议题。

而对于Java SE5所具有的每一个重大的新特性，本书都有一个新的章节与之对应。其他的新特性则加入到了原有的章节中。我还一直在研究设计模式，因此在本书中，也介绍了设计模式的相关内容。

本书经历了重大的重组。这大多源自教授Java的过程，以及我对于“章节”的意义的重新思考。以前，我会不假思索地认为，每个“章节”应该包含一个“足够大的”主题。但是，在我教授设计模式的时候，我发现，如果每次只介绍一个模式（即使讲课的时间很短），然后立刻组织大家做练习，此时那些学员们的表现是最好的（我发现，这种节奏对于我这个老师而言也更有乐趣）。因此，在这一版中，我试着打破按主题划分章节的做法，也不理会章节的长度。我想，这也是一个改进。

4

我同样也认识到代码测试的重要性。必须要有一个内建的测试框架，并且每次你开发系统时都必须进行测试。否则，根本没有办法知道代码可靠与否。为了做到这一点，我开发了一个测试框架以显示和验证本书中每一个程序的输出结果。（该框架是用Python编写的，你可以在www.MindView.net找到可下载的代码。）关于测试的话题在附录中有讨论，你可以在http://MindView.net/Books/BetterJava找到。其中还包含了其他一些基本技术，我认为所有程序员都应该将它们加入到自己的工具箱中。

此外，我还仔细检查了书中的每一个示例，并且问我自己，“我为什么采用这种方式实现？”对大多数的示例，我都做了一定程度的修订与改进，使得这些示例更加贴切。同时，也传达出我所认为的Java编程中的最佳实践（至少起到抛砖引玉的作用）。许多以前的示例都经过了重新设计与重新编写，同时，删除了不再有意义的示例，也添加了新的示例。

读者们为此书的前三个版本提出了许多许多精彩的意见。这自然使我觉得非常高兴。不过，偶尔读者也会有抱怨，例如有读者埋怨“本书太长了”。对我而言，如果“页数太多”是你唯一的苦恼，那这真令人哭笑不得。（据说奥地利皇帝曾抱怨莫扎特的音乐“音符太多”！我可不是想把自己比作莫扎特。）此外，我只能猜测，发出这种抱怨的读者还不了解Java语言的博大精深，而且也没有看过这一领域的其他书籍。无论如何，在这一版中，我已经删减了过时无用，或不再重要的内容。总的来说，我已经尽我所能仔细复查了全书，进行了必要的增删与改进。对于删除旧的章节，我还是挺放心的。因为原始的材料在网站上都有（www.MindView.net）。本书从第一版到第三版，以及本书的附录，都可以从此网站上下载。

5

对于仍然不能接受本书篇幅的读者，我向你们道歉。请相信，我已经尽我所能精简本书的长度了。

封面图片的故事

《Thinking in Java》的封面创作灵感来自于美国的 Arts & Crafts运动。该运动始于世纪之交，

并在1900 到1920年间达到顶峰。它起源于英格兰，是对工业革命带来的机器产品和维多利亚时代高度装饰化风格的回应。Arts & Crafts强调简洁设计，而回归自然是其整个运动的核心，注重手工制造及推崇个性化设计，可是它并不回避使用现代工具。这和我们现今的情形有很多相似之处：世纪之交，从计算机革命的最初起源到对个人来说更精简、更意味深长的事物的演变，以及对软件开发技能而不仅是生产程序代码的强调。

我以同样的眼光看待Java：尝试将程序员从操作系统机制中解放出来，朝着“软件艺师”的方向发展。

我和封面设计者自孩提时代就是朋友，我们从这次运动中获得灵感，并且都拥有源自那个时期的（或受那个时期启发而创作的）家具、台灯和其他作品。

这个封面暗示的另一主题是一个收集盒，博物学家可以用它来展示他们保存的昆虫标本。这些昆虫可以看作是对象，并放置到“盒”这个对象当中，而盒对象又放置到“封面对象”当中，这形象地说明了面向对象程序设计中最为基本的“集合”概念。当然，程序员可能会不禁联想到“程序缺陷（bug）”；这些虫子被捕获，并假设在标本罐中被杀死，最后禁闭于一个展示盒中，似乎暗示Java有能力发现、显示和制服程序缺陷（事实上，这也是它最为强大的属性之一）。

在本版中，我创造了一幅水彩画，你可以在封面的背景中看到它。 6

致谢

首先感谢和我一起开研讨课、提供咨询和开发教学计划的这些合作者：Dave Bartlett、Bill Venners、Chuck Allison、Jeremy Meyer和 Jamie King。在我转而不停地竭力为那些像我们一样的独立人群开发在一起协同工作的最佳模式的时候，你们的耐心让我感激不已。

最近，无疑是因为有了Internet，我可以和极其众多的人一起合作，他们协助我一起努力，他们通常是在家办公。过去，我可能必须为这些人提供相当大的办公空间，不过由于现在有了网络、传真以及电话，我不需要额外的开销就可以从他们的帮助中受益。在我尽力学习更好地与其他人相处的过程中，你们都对我很有帮助，并且我希望继续学习怎样使我的工作能够通过借鉴他人的成果而变得更出色。Paula Steuer在接管我偶尔的商务活动时发挥了不可估量的价值，他使它们变得井井有条（Paula，感谢你在我懈怠时对我的鞭笞）。Jonathan Wilcox, Esq.详细审视了我公司的组织结构，推翻了每一块可能隐藏祸害的石头，并且使所有事情都条理化和合法化了，这让我们心服口服。感谢你的细心和耐心。Sharlynn Cobaugh使自己成为声音处理的专家，她是创建多媒体培训CD ROM和解决其他问题的精英成员之一。感谢你在面临难于处理的计算机问题时的坚定不移。在布拉格Amaio的人们也提出了一些方案来帮助我。Daniel Will-Harris最先受到在网上工作的启发，因此他当然是我所有设计方案的主要人物。

多年以来，Gernal Weinberg通过他的学术会议和研讨会，已经成为了我非正式的教练和导师，我十分感谢他。

Ervin Varga在第4版的技术纠正方面提供了巨大的帮助——尽管其他人在各个章节和示例方面也帮助良多，但是Ervin是本书最主要的技术复审者，他还承担了第4版的解决方案指南的重写任务。Ervin发现的错误和对本书所作的完善对本书来说价值连城。他对细节的投入和关注程度令人惊异，他是我所见过的远远超过其他人的最好的技术读者。感谢你，Ervin。 7

我在Bill Venners的www.Artima.com上的weblog，已经成为了当我需要交流思想时的一种解决之道。感谢那些通过提交评论帮助我澄清概念的人们，包括James Watson、Howard Lovatt、

Michael Barker以及其他一些人，特别是那些在泛型方面提供帮助的人。

感谢Mark Welsh不懈的帮助。

Evan Cofsky一如既往地提供了有力的支持，他埋头处理了大量晦涩的细节，从而建立和维护了基于Linux的Web服务器，并保持MindView服务器始终处于协调和安全的状态。

一份特别的感谢要送给我的新朋友，咖啡，它为本项目产生了几乎无穷无尽的热情。当人们来到MindView研讨课时，科罗拉多州Crested Butte的Camp4 Coffee已经成为了标准住所，并且在研讨课中间休息期间，它是我所遇到的最好的饮食场所。感谢我的密友Al Smith，是他使这里成为如此好的一个地方，成为Crested Butte培训期间一个如此有趣和愉快的场所。还要感谢Camp4的所有泡吧常客们，很高兴他们总是为我们提供一些饮料。

感谢Prentice Hall的人们不断地为我提供我所需要的一切，并容忍我所有的特殊需求，而且不厌其烦地帮我把所有事情都搞定。

在我的开发过程中，有些工具已经被证明是无价的；但每次使用它们时都会非常感激它们的创建者。Cygwin (http://www.cygwin.com)为我解决了无数Windows不能解决的问题，并且每天我都会变得更加依赖它（如果在15年前当我的头脑因使用Gnu Emacs而搞得发懵的时候，能有这些该多好啊）。IBM的Eclipse (http://www.eclipse.org)对开发社区做出了真正杰出的贡献，并且随着它的不断升级，我期望能看到它的更伟大之处（IBM是怎样成为潮流所向的？我肯定错过了一份备忘录）。而JetBrains IntelliJ Idea则继续开阔着开发工具的创新之路。

我一开始就将Sparxsystems的Enterprise Architecture用于本书，并且它很快就成为了我选择的UML工具。Marco Hunsicker的Jalopy代码格式化器（www.triemax.com）在大量的场合都派上了用场，而且Marco在将其配置成满足我的特殊需求方面也提供了大量的帮助。我还发现Slava Pestov的JEdit及其插件经常会显得很有用（www.jedit.org），并且对于研讨课来说，它是非常适合初学者的编辑器。

当然，如果我在其他地方强调得还不够的话，我得再次重申，我经常使用Python（www.Python.org）解决问题，在我的密友Guido Van Rossum和PythonLabs那些身材臃肿愚笨的天才人物的智慧结晶的基础上，我花费了好几天的时间进行冲刺（Tim Peters，我现在已经把你借的鼠标加了个框，正式命名为TimBotMouse）。你们这伙人必须到更健康的地方去吃午餐。（还要感谢整个Python社区，他们是一帮令人吃惊的群体。）

很多人向我发送修正意见，我感激所有这些人，第1版特别要感谢：Kevin Raulerson（发现无数的程序缺陷），Bob Resendes（简直难以置信），John Pinto、Joe Dante、Joe Sharp（三位都难以置信），David Combs（校正了许多语法和声明），Dr. Robert Stephenson、John Cook、Franklin Chen、Zev Griner、David Karr、Leander A. Stroschein、Steve Clark、Charles A、Lee、Austin Maher、Dennis P. Roth、Roque Oliveira、Douglas Dunn、Dejan Ristic、Neil Galarneau、David B. Malkovsky、Steve Wilkinson以及许许多多的人。本书第1版在欧洲发行时，Marc Meurrens在电子版宣传和制作方面做出了巨大的努力。

感谢在本书第2版中使用Swing类库帮助我重新编写示例的人们，以及其他助手—Jon Shvarts、Thomas Kirsch、Rahim Adatia、Rajesh Jain、Ravi Manthena、Banu Rajamani、Jens Brandt、Nitin Shivaram、Malcolm Davis，还有所有表示支持的人。

在第4版中，Chris Grindstaff对SWT一节的撰写提供很多帮助，而Sean Neville为我撰写了Flex一节的第一稿。

每当我认为我已经理解了并发编程时，又会有新的奇山险峰等待我去征服。感谢Brian Goetz帮助我克服了在撰写新版本的“并发”一章时遇到的种种艰难险阻，并发现了其中所有的缺陷（我希望如此！）

对Delphi的理解使我更容易理解Java，这一点儿都不奇怪，因为它们有许多概念和语言设计决策是相通的。我的懂Delphi的朋友们给我提供了许多帮助，使我能够洞察一些非凡的编程环境。 9
他们是Marco Cantu（另一个意大利人-难道会说拉丁语的人在学习Java时有得天独厚的优势？）、Neil Rubenking（直到发现喜欢计算机之前，他一直都在做瑜珈／素食／禅道），当然还有Zack Urlocker（最初的Delphi产品经理），他是我游历世界时的好伙伴。我们都很感激Anders Hejlsberg的卓越才华，他在C#领域不懈地奋斗着（正如你将在本书中看到的，C#是 Java SE5主要的灵感之一）。

我的朋友Richard Hale Shaw（以及Kim）的洞察力和支持都很有帮助。Richard和我花了数月时间将教学内容合并到一起，并为参加学习的学生设计出一套完美的学习体验。

书籍设计、封面设计以及封面照片是由我的朋友Daniel Will-Harris制作的。他是一位著名的作家和设计家（http://www.WillHarris.com），在计算机和桌面排版发明之前，他在初中的时候就常常摆弄刮擦信（rub-on letter），他总是抱怨我的代数含糊不清。然而，要声明的是，是我自己制作的照排好的（camera-ready）页面，所以所有排字错误都应该算到我这里。我是用Microsoft Word XP for Windows来编写这本书的，并使用Adobe Acrobat制作照排页面的。本书是直接从Acrobat PDF文件创建而来的。电子时代给我们带来了厚礼，我恰巧是在海外创作了本书第1版和第2版的最终稿—第1版是在南非的开普敦送出的，而第2版却是在布拉格寄出的。第3版和第4版则来自科罗拉多州的Crested Butte。正文字体是Georgia，而标题是Verdana。封面字体是ITC Rennie Machintosh。

特别感谢我的所有老师和我的所有学生（他们也是我的老师）。

Molly，在我从事这一版的写作时总是坐在我腿上，为我提供了她特有的温软而毛茸茸的支持。

曾向我提供过支持的朋友包括（当然还不止他们）：Patty Gast(Masseuse extraordinary)，Andrew Binstock，SteveSinofsky，JD Hildebrandt，Tom Keffer，Brian McElhinney，Brinkley Barr，《Midnight Engineering》杂志社的Bill Gates，Larry Constantine和Lucy Lockwood，Gene Wang，Dave Mayer，David Intersimone，Chris和Laura Strand，Almquists，Brad Jerbic，Marilyn Cvitanic，Mark Mabry，Dave Stoner，Cranstons，Larry Fogg，Mike Sequeira，Gary Entsminger，Kevin 和 10
Sonda Donovan，Joe Lordi，Dave和Brenda Bartlett，Patti Gast，Blake，Annette&Jade， 11～12
Rentschlers，Sudeks，Dick，Patty和Lee Eckel，Lynn和Todd以及他们的家人。当然还有我的父亲和母亲。

绪　论

“上帝赋予人类说话的能力，而言语又创造了思想，思想是人类对宇宙的量度。”

——摘自《Prometheus Unbound》，Shelley

人类……极其受那些已经成为社会表达工具的特定语言的支配。想像一下，如果一个人可以不使用语言就能够从本质上适应现实世界，语言仅仅是解决具体的交流和反映问题时偶尔才用到的方式，我们会发现，这只能是一种幻想。事实上，“真实世界”在很大程度上是不知不觉地基于群体的语言习惯形成的。

——摘自《The Status of Linguistics As A Science》，1929, Edward Sapir

如同任何人类语言一样，Java提供了一种表达概念的方式。如果使用得当，随着问题变得更庞大更复杂，这种表达工具将会比别的可供选择的语言更为简单、灵活。

我们不应该将Java仅仅看作是一些特性的集合——有一些特性在孤立状态下没有任何意义。只有在考虑到设计，而不仅仅是编码时，才能完整地运用Java的各部分。而且，要按照这种方式来理解Java，必须理解在语言和编程中经常碰到的问题。这本书讨论的是编程问题，它们为什么成为问题，以及Java已经采取什么样的方案来解决它们。因此，每章所阐述的特性集，都是基于我所看到的这一语言在解决特定类型问题时的方式。按照这种方式，我希望能够每次引导读者前进一点，直到Java思想意识成为你最自然不过的语言。

自始至终，我一直持这样的观点：你需要在头脑中创建一个模型，以加强对这种语言的深入理解；如果你遇到了疑问，就将它反馈到头脑中的模型并推断出答案。

前提条件

本书假定你对程序设计有一定程度的熟悉：你已经知道程序是一些语句的集合，知道子程序/函数/宏的概念，知道像“if”这样的控制语句和像“while”这样的循环结构，等等。不过，你可能在许多地方已经学到过这些，例如使用宏语言进行程序设计，或者使用像Perl这样的工具工作。只要你已经达到能够自如地运用程序设计基本思想的程度，你就能够顺利阅读本书。当然，本书对C程序员来说更容易，对于C++程序员更是如此，但是，即使你没有实践过这两种语言，也不要否定自己—而应该更加努力学习。并且，从www.MindView.net处可下载的《Thinking in C》多媒体研讨课能够带领你快速学习所必需的Java基础知识）。不过，我还会介绍面向对象（OOP）的概念和Java的基本控制机制。

尽管本书可能会经常引用、参考C和C++语言的特性，但这并不是打算让它们成为内部注释，而是要帮助所有的程序员正确看待这些语言，毕竟Java是从这些语言衍生而来的。我会努力简化这些引用、参考，并且对那些我认为一个非C/C++程序员可能不太熟悉的地方加以解释。

学习Java

大概在我的第一本书《Using C++》（Osborne/McGraw-Hill，1989）出版发行的同一时候，我

就开始教授这种语言了。讲授程序设计语言已经成为我的职业了；自1987年以来，我在世界各地的听众中看到，有的昏昏欲睡，有的面无表情，有的表情迷茫。当我开始给一些小团体进行室内培训时，在这些实践中我发现了一些事情。即使那些面带微笑频频点头的人也对很多问题心存困惑。我发现，多年来在软件开发会议上由我主持的C++分组讨论会（后来变成Java分组讨论会）中，我和其他的演讲者往往是在极短的时间内告诉听众许多话题。因此，最后由于听众的水平不同和讲授教材的方式这两方面的原因，我可能最终会失去一部分听众。可能这样要求得太多了，但因为我是传统演讲的反对者之一（而且对于大多数人来说，我相信这种抵制是因为厌倦），因此我想尽力让每个人都可以跟得上演讲的进度。 14

我曾经一度在相当短的时间内做了一系列不同的演讲。因此，我结束了“实践和迭代”（一项在Java程序设计中也得到很好运用的技术）的学习。最后，我根据自己在教学实践中学到的东西发展出一门课程。我的公司—MindView有限公司现在提供公开的室内“Thinking in Java”研讨课；这是我们主要的初级研讨课，为以后更高级的研讨课提供基础。读者可以到网站www.MindView.net上了解详细情况。（初级研讨课在“Hands-On Java”光盘上也能找到。上述网站也可以找到相关信息。）

从每个研讨课获得的反馈信息都可以帮助我去修改和重新制定课程教材，直到我认为它能够成为一个良性运转的教学工具为止。不过不能将本书视为一般的研讨课笔记；我努力在本书中放入尽可能多的信息，并且合理地组织本书结构，从而引导读者顺利进入下一主题。最重要的是，本书面向那些孤军奋战一门新的程序设计语言的读者。

目标

就像我前一本书《Thinking in C++》那样，在设计本书时，我脑子里始终思考的一件事情就是：人们学习语言的方式。当我思索书中的一章时，我思索的是如何在研讨课上教好一堂课。研讨课听众的反馈意见帮助我理解了哪些是需要详细阐明的有难度的部分。在某些领域，我一开始雄心勃勃，在其中一下子囊括了过多的特性，后来通过在讲解这些材料的过程中，我逐渐意识到如果要囊括过多的特性，就必须对它们全部解释清楚，而这很容易使学生产生混淆。

因此，本书的每一章都设法只教授一个特性，或者一小组互相关联的特性，并且不会依赖于还未介绍的概念。通过这种方式，你可以在你当前所掌握的知识背景下，在继续向前学习之前，消化吸收每一部分内容。

在这本书中我想达到的目标是：

1) 每一次只演示一个步骤的材料，以便读者在继续后面的学习之前可以很容易地消化吸收每一个观念。仔细地对特性的讲解进行排序，以使得你在看到对某个特性的运用之前，会先了解它。当然，这并非总是可行的，在那些不可行的情况下，会给出一个简短的介绍性描述。 15

2) 使用的示例尽可能简单、短小。这样做有时会妨碍我们解决“真实世界”的问题，但是，我发现对于初学者，能够理解例子的每一个细节，而不是理解它所能够解决的问题范畴，前者通常更能为他们带来愉悦。同样，适合在教室内学习的代码数量也有严格限制。正因为如此，我将毫无疑问地会遭到批评-批评我使用“玩具般的示例”，但是我乐意接受那些有利于为教育带来益处的种种事物。

3) 向读者提供“我认为对理解这种程序设计语言来说很重要”的部分，而不是提供我所知道的所有事情。我相信信息在重要性上存在层次差别，有一些事实对于95%的程序员来说永远

不必知道—那些只会困扰他们并且使他们对程序复杂性平添许多感触。举一个C语言的例子，如果能够记住操作符优先表（我从未记住过），那么可以写出灵巧的代码。但是你要再想一想，这样做会给读者/维护者带来困惑。因此忘掉优先权，在不是很清楚的时候使用圆括号就行了。

4) 使每部分的重点足够明确，以便缩短教学和练习之间的时间。这样做不仅使听众在亲身参与研讨课时思维更为活跃和集中，而且还可以让读者更具有成就感。

5) 给读者打下坚实的基础，使读者能够充分理解问题，以便转入更难的课程学习和书籍阅读中。

根据本书教学

本书最初的版本是从一个为期一周的研讨课演变而来的，当时Java还处于初级阶段，因此一周已经足以覆盖Java的语言特性了。随着Java的成长，有越来越多的特性和类库不断地添加了进来，我固执地试图仍旧在一周内教授所有的内容。那时，有一位顾客请我讲课，内容“只包括基础知识”，我教授的过程中，我发现在一周的时间内填鸭式的教授所有的内容，对于我自己和参加研讨课的人来说，都是一种痛苦。Java已经不再是一种可以在一周内教授的“简单”语言了。

16

这份精力和感悟在极大程度上促使我对本书进行了重新的组织，现在它已经被设计为可以支撑一个两周的研讨课，或者是一门两学期的大学课程。介绍性的部分在“通过异常处理错误”一章就结束了，但是你可能还想补充了解一些对JDBC、Servlet和JSP的介绍，这些内容构成了另外一门基础课程，即Hands-on Java 光盘的核心内容。本书剩余部分可以组成一门中级课程，即Intermediate Thinking in Java光盘中所包含的材料。这两张光盘在www.MindView.net都有售。

- 通过www.prenhallprofessional.com与Prentice-Hall联系，可以得到能够教授本书这些材料的教师信息。

JDK的HTML文档

Sun公司的Java语言及其类库（可以从java.sun.com免费下载）配套提供了电子版文档，可使用Web浏览器阅读。许多出版的Java书籍中也都有这份文档的备份。你可能已经拥有了它，或者能够下载；所以除非必要，本书不会再重复那份文档。因为一般来说，用Web浏览器查找类的描述比在书中查找要快得多（并且在线文档更能保持更新）。你仅需要参考“JDK文档”。只有当需要对文档进行补充，以便你能够理解特定实例时，本书才会提供有关类的一些附加说明。

练习

在研讨课上，我发现一些简单的练习非常利于学生们理解掌握有关概念，因此在每一章的最后都安排了一些习题。

大多数练习设计得都很简单，可以让学生在课堂上在合理的时间内完成这些作业，以便指导老师检查辅导以确保所有的学生都吸收了教材的内容。有一些题目具有挑战性，但并没有难度很高的题目。

一些经过挑选的练习答案可以在名为The Thinking in Java Annotated Solution Guide的电子文档中找到，仅需少许费用便可以从www.MindView.net下载得到。

17

Java基础

本书还附送可从www.MindView.net处下载的免费的多媒体研讨课。这是《Thinking in C》的研讨课，它介绍了Java语法沿用的C语言中的语法、操作符及函数。在本书以前的版本中，这部分内容收录在随书附送的“Java基础”CD中，但是现在这个研讨课可以免费下载了。

我原本打算让Chuck Allison把“Thinking in C”创建成一个独立产品，不过我还是决定将它和第2版的《Thinking in C++》，以及第2版和第3版的《Thinking in Java》包含在一起，这样做是为了让参加研讨课的、没有太多C语言基本语法背景的人们能够很方便地找到相关资料。应该抛开这种思想：“我是一个聪明的程序员，我不想学习C，而想学习C++或Java，因此我会跳过C直接到C++/Java。”在到了研讨课上后，这些人渐渐明白，很好地理解C语言语法这个先决条件很必要。

技术在不断发生变化，将《Thinking in C》重新制作为可下载的Flash形式比将其收录在CD中要更具实际意义。通过在线提供这个研讨课，我可以保证每个人都可以事先做好充足的准备。

《Thinking in C》研讨课也让本书获得了更多的读者。尽管本书中“操作符”和“控制执行流程”两章覆盖了Java继承自C的基本部分，但是在线研讨课仍旧是更好的介绍，而且它要求学生所具备的程序设计背景比这本书要求的还要少。

源代码

本书的所有源代码都能以保留版权的免费软件的形式得到，它们是以单一包的形式发布的，访问www.MindView.net网站便可获取。为了确保你获得的是最新版本，这个发布这些源代码和本书电子版的网站是一个官方网站。你可以在课堂或其他教育场所发布这些代码。

保留版权的主要目的是为了确保源代码能够被正确地引用，并且防止在未经许可的情况下，在出版媒体中重新发布这些代码（只要说明是引用了这些代码，那么在大多数媒介中使用本书中的示例通常不是问题）。 18

在每个源码文件中，都包含下述版权声明文字：

```
//:! Copyright.txt
This computer source code is Copyright ©2006 MindView, Inc.
All Rights Reserved.

Permission to use, copy, modify, and distribute this
computer source code (Source Code) and its documentation
without fee and without a written agreement for the
purposes set forth below is hereby granted, provided that
the above copyright notice, this paragraph and the
following five numbered paragraphs appear in all copies.

1. Permission is granted to compile the Source Code and to
include the compiled code, in executable format only, in
personal and commercial software programs.

2. Permission is granted to use the Source Code without
modification in classroom situations, including in
presentation materials, provided that the book "Thinking in
Java" is cited as the origin.

3. Permission to incorporate the Source Code into printed
media may be obtained by contacting:

MindView, Inc. 5343 Valle Vista La Mesa, California 91941
```

```
Wayne@MindView.net

4. The Source Code and documentation are copyrighted by
MindView, Inc. The Source code is provided without express
or implied warranty of any kind, including any implied
warranty of merchantability, fitness for a particular
purpose or non-infringement. MindView, Inc. does not
warrant that the operation of any program that includes the
Source Code will be uninterrupted or error-free. MindView,
Inc. makes no representation about the suitability of the
Source Code or of any software that includes the Source
Code for any purpose. The entire risk as to the quality
and performance of any program that includes the Source
Code is with the user of the Source Code. The user
understands that the Source Code was developed for research
and instructional purposes and is advised not to rely
exclusively for any reason on the Source Code or any
program that includes the Source Code. Should the Source
Code or any resulting software prove defective, the user
assumes the cost of all necessary servicing, repair, or
correction.

5. IN NO EVENT SHALL MINDVIEW, INC., OR ITS PUBLISHER BE
LIABLE TO ANY PARTY UNDER ANY LEGAL THEORY FOR DIRECT,
INDIRECT, SPECIAL, INCIDENTAL, OR CONSEQUENTIAL DAMAGES,
INCLUDING LOST PROFITS, BUSINESS INTERRUPTION, LOSS OF
BUSINESS INFORMATION, OR ANY OTHER PECUNIARY LOSS, OR FOR
PERSONAL INJURIES, ARISING OUT OF THE USE OF THIS SOURCE
CODE AND ITS DOCUMENTATION, OR ARISING OUT OF THE INABILITY
TO USE ANY RESULTING PROGRAM, EVEN IF MINDVIEW, INC., OR
ITS PUBLISHER HAS BEEN ADVISED OF THE POSSIBILITY OF SUCH
DAMAGE. MINDVIEW, INC. SPECIFICALLY DISCLAIMS ANY
WARRANTIES, INCLUDING, BUT NOT LIMITED TO, THE IMPLIED
WARRANTIES OF MERCHANTABILITY AND FITNESS FOR A PARTICULAR
PURPOSE. THE SOURCE CODE AND DOCUMENTATION PROVIDED
HEREUNDER IS ON AN "AS IS" BASIS, WITHOUT ANY ACCOMPANYING
SERVICES FROM MINDVIEW, INC., AND MINDVIEW, INC. HAS NO
OBLIGATIONS TO PROVIDE MAINTENANCE, SUPPORT, UPDATES,
ENHANCEMENTS, OR MODIFICATIONS.

Please note that MindView, Inc. maintains a Web site which
is the sole distribution point for electronic copies of the
Source Code, http://www.MindView.net (and official mirror
sites), where it is freely available under the terms stated
above.

If you think you've found an error in the Source Code,
please submit a correction using the feedback system that
you will find at http://www.MindView.net.
///:~
```

19

你可以在自己的项目中引用这些代码，也可以在课堂上引用它们（包括你的演示材料），只要保留每个源文件中出现的保留版权声明即可。

20

编码标准

在本书的正文中，标识符（方法、变量和类名）排为粗体。大多数关键字也排为粗体，但是不包括那些频繁使用的关键字，例如“class”，因为如果将它们也设为粗体会令人十分厌烦。

对于本书中的示例，我使用了一种特定的编码格式，此格式尽可能地遵循了Sun自己在所有代码中实际使用的格式，在它的网站上你会发现这些代码（见java.sun.com/docs/codeconv/index.html），并且似乎大多数Java开发环境都支持这种格式。如果你已经读过我的其他著作，你会注意到Sun的编码格式与我的一致-尽管这与我没什么关系（我了解这一点），但我还是很高兴。

对代码进行格式化这个议题常常会招致几个小时的热烈争论，因此我不会试图通过自己的示例来规定正确的格式；我对自己使用的格式有自己的想法。因为Java是一种自由形式的程序设计语言，所以你可以继续使用自己喜欢的格式。编码风格问题的一种解决方案是使用像Jalopy(www.triemax.com)这样的工具来将格式转变为适合你的形式，该工具帮助我撰写了此书。

本书中打印的代码文件都用一个自动系统进行过测试，应该全部都能够运行，而且无编译错误。

本书聚焦于Java SE5/6，并用它们进行过测试。如果你需要学习本书这一版中没有讨论的Java语言的先前版本，可以从www.MindView.net处免费下载本书的第1版到第3版。

错误

无论作者使用多少技巧去查找错误，但是有些错误还是悄悄地潜藏了起来，并且经常对新读者造成困扰。如果你发现了任何你确信是错误的东西，请使用在www.MindView.net处可以找到的为本书专设的链接来提交错误以及你建议的修正。对你的帮助我将不胜感激。

21
~
22